LLM Eval CI/CD Workstation

THE COMPREHENSIVE LLMOPS QUALITY GATEWAY DESIGN SPECIFICATION

1. Project Importance & Executive Summary

In software engineering, reliability is maintained via automated unit tests (e.g. Jest, PyTest) that verify if code changes break existing functionality. In artificial intelligence and LLM application development, this safety net has historically been missing. Large Language Models are non-deterministic: modifying a prompt template, upgrading a base model, or altering chunking variables in a RAG database can introduce unexpected bugs, incorrect assumptions, or hallucinations. This is a massive blocker for enterprise production adoption.

This project solves the LLM reliability crisis by introducing an automated LLMOps Evaluation CI/CD Pipeline. It serves as an automated gatekeeper. When developers modify prompts or swap engines (such as using Google Gemini 1.5 Flash), the system automatically compiles and tests their configuration against a benchmark golden dataset of 100+ items. It measures faithfulness, hallucination rates, cost, and latency, blocking merges immediately if any Quality Gate SLA is breached.

By combining backend command-line gatekeepers with a premium interactive workstation dashboard, this system provides developers with real-time insight into prompt performance trends, visual regression comparators, and node-level details inspectors.

2. System Connection & Architecture Map

3. Detailed Component & Tool Breakdown

scripts/eval-runner.js (Automated Gatekeeper)

A Node.js CLI script designed to run inside automated environments (such as Git Hooks or GitHub Actions). It parses command-line flags, pulls prompt templates, retrieves benchmark context chunks, triggers LLM evaluation queries, evaluates faithfulness metrics, appends results to the history database, and exits with non-zero error codes on quality gate failures.

src/data/golden_dataset.json (Golden Benchmark)

A dataset of 100+ standard question-answer pairs spanning four core categories: Customer Support, Technical Coding, Financial Extraction, and Legal Compliance. Each case contains expectations and context bounds to test prompt limits.

src/data/runs_history.json (Run Log Database)

Acts as the history ledger. It stores logs of every commit, including the author, prompt template, chunk setup, overall metrics, and a case-by-case evaluation audit trail. This serves as the data layer for the frontend dashboard.

src/App.jsx & index.css (Cockpit Frontend)

A premium, high-fidelity developer cockpit built using Vite, React, Recharts, and custom CSS variables. It replaces standard text headers with radial progress meters, slide-out node inspector drawers, a commit diff comparator, and a real-time console terminal to test prompt updates with live Gemini API calls.

4. Quality Gates & SLA Metrics scoring

To prevent bad prompts from going to production, the gatekeeper enforces three main Service Level Agreements (SLAs). If any run breaches these thresholds, the pipeline fails:

Hallucination Rate (SLA: <= 5%)

Calculates the proportion of assertions made by the LLM that cannot be justified by the reference context. The system scans the generated answer, extracts claims, and verifies if the key entities and facts are grounded inside the context. If the hallucination rate exceeds 5%, the build is flagged as unstable.

Faithfulness (SLA: > 90%)

Grades the degree of reliance of the response on the context. If the model answers the question using external pre-trained knowledge rather than using the provided documentation, the faithfulness score drops. High-quality systems require the model to rely strictly on retrieved chunks.

p95 Response Latency (SLA: <= 2000ms)

Ensures the 95th percentile response time is under 2 seconds. A prompt that is too long or retrieves too many docs (high top-K) slows down response times. Measuring p95 protects production from latency spikes.

Answer Relevancy (SLA: Informational)

Analyzes keyword overlap and semantic matching between the generated answer and expected ground truth. This ensures prompt adjustments did not alter the intended answer formatting or style.

Eval Cost (SLA: Informational)

Calculates the API cost based on input/output token counts and model token pricing structures, preventing sudden cost regressions.

5. User Guide: How to Use & Run

This workstation is designed for both local prompt development and automated pipeline testing.

1. Setting Up Google Gemini API Key

To perform real LLM audits, navigate to the "CI Run Simulator" tab, choose the "Gemini 1.5 Flash (REAL API - Free Tier)" model, and enter your free API Key from Google AI Studio. Note that the key starts with "AIzaSy" and is stored securely only in your browser's local storage.

2. Running Interactive Prompt Tests

Under the simulator tab, you can customize prompt templates (using {{context}} and {{question}} placeholders) and RAG chunk configurations. Adjust the "Evaluation Subset" to 10 cases to prevent API rate limits, and click "Commit Change & Run Pipeline". The retro terminal console will output a live log, showing test results step-by-step.

3. Auditing Regressions & Differences

Navigate to the "Dual-Run Comparator" tab and select two different commits. The dashboard will display a side-by-side comparison of the prompts and config settings alongside difference metrics, showing you exactly where the system degraded.

6. Chronological Development Log: Steps Taken

This details the exact step-by-step implementation sequence followed to build this workstation from scratch:

Step 1: Diagnostics & Directory Initialization **[Tool: Terminal, Node v26.4.0, npm v11.17.0]**
Inspected environment permissions and package support. Initialized the Vite-React project structure in the workspace.

Step 2: Golden Dataset Generation **[Tool: scripts/generate-dataset.js, Node FS modules]**
Wrote a script to programmatically compile exactly 100 benchmark Q&A pairs covering Customer Support, Technical Coding, Finance, and Legal categories with expected answers and contexts.

Step 3: Seeding Runs History Database **[Tool: scripts/generate-history.js]**
Generated a JSON database with 5 baseline historical runs. Seeding random variations in latency and hallucination rates created realistic regression data for charts.

Step 4: Writing the CLI Pipeline Runner **[Tool: scripts/eval-runner.js, process.argv modules]**
Programmed the pipeline executor. Implemented terminal logs, SLA validation algorithms, and process exit codes (0/1) to block merge operations.

Step 5: Styling System & Component Coding **[Tool: src/index.css, src/App.jsx, Recharts, Lucide React]**
Built the premium glassmorphism dark-theme sidebar navigation. Coded circular progress rings, searchable lists, and an inspect drawer.

Step 6: Gemini API Integration & Build **[Tool: Vite Compiler, Google Gemini REST API Endpoint]**
Configured App.jsx to securely capture the Gemini API key and perform live fetch requests. Run "npm run build" to check compilation.

Step 7: Cloud Deployment to Vercel **[Tool: npx vercel, Vercel Device Auth Flow]**
Logged into Vercel Hobby tier using remote browser device authorization and published the workstation live to solar-six-tan.vercel.app.

7. Transparency Log: Real vs. Simulated Components

When presenting this system in technical interviews (such as with Microsoft), complete transparency regarding implementation parameters is essential. Below is the exact matrix of simulated versus real components:

1. Model Inference and API Responses

Local simulation models (such as gpt-4o, claude-3-5-sonnet) run locally without calling third-party servers. They simulate responses using statistical distributions. Conversely, the gemini-1.5-flash path executes actual HTTP POST queries to Google's API endpoint, producing live outputs.

2. Correctness and Faithfulness Grading

Simulated runs assign mock percentage points based on hyperparameter correlations (e.g. failing faithfulness when chunk size is too small). Real Gemini evaluations run a dual-chain call: first generating the answer, then calling Gemini a second time as a judge to evaluate faithfulness and output a parsed JSON block.

3. API Error Handling and Pipeline Aborts

The mock path bypasses network states for local testing. The live Gemini path is designed to fail loudly and visibly. It intercepts HTTP error payloads (such as invalid API authorization or quota exceptions), prints the failure in the terminal, aborts the pipeline execution, and returns exit code 1 to block merges.